Argdown

Some Backgroundinformation about Argdown

John Doe
Institute of Something

2026-04-24



What is Argdown? The Word Analogy

	Text world	Argument map world
Format/Syntax	Docx format for texts	Argdown format for argument maps
Artefacts/Products	Written text	Argument map / dialectical structure
Tools to create/edit/export the artefacts	Word, LibreOffice	VS Code + Argdown Extension, Editor, Argdown Sandbox, Argdown CLI

Comparison of Argdown Tools

1. VSCode + Argdown Extension
    - Collaboration via Live Share Extension. (Also useful for live reconstructions and collaborative exercises with students.)
2. VS Code Web:
    - Extensions can be installed “permanently”. (https://code.visualstudio.com/docs/setup/vscode-web#_extensions)
    - Opening local folders: not supported in Firefox (but works in Chrome and Edge) (https://code.visualstudio.com/docs/setup/vscode-web#_file-system-api)
3. Argdown Sandbox: Runs directly in the browser.
    - Save and load only via copy & paste.
4. Argdown CLI
    - Command-line tool: export to many formats (besides image files: json, graphml, dot, html)

Summary

	Sandbox	VSCode	VSCode Web	Argdown CLI
Syntax Highlighting	✅	✅	✅	➖
Installation	🤩	😕	🙂	😢
Collaboration	❌	☑️	☑️	➖
Code Completion	❌	✅[1]	✅[2]	➖

Legend:

✅: Included automatically

☑️: Possible in principle (but requires additional software)

❌: Not available

➖: Category not applicable

1. Not inside Argdown code blocks
2. Not inside Argdown code blocks



Graph Layout

👉 Argument maps use layout algorithms to position nodes and edges.

Options for manual layout adjustments:

- Manual layout:
  - Export to graphml and edit in other programs (e.g. yEd).
  - Export to dot and use all customisation options of Graphviz.
- Within Argdown:
  - Set Graphviz settings: https://argdown.org/guide/changing-the-graph-layout.html#layouting-with-viz-js

Representing Dialectical Structures

argdown

Map

```
===
selection:
    excludeDisconnected: false
===

<Argument with intermediate conclusions – one node>

(1) First premise
(2) Second premise
----
(3) *Therefore:* Intermediate conclusion
(4) One more premise.
----
(5) *Therefore:* Conclusion.
```

By default, disconnected nodes are not shown.[1]

1. See https://argdown.org/guide/creating-statement-and-argument-nodes.html#the-excludedisconnected-selection-setting



Representing Dialectical Structures

Intermediate conclusion and final conclusion defined as independent statements.

argdown

Map

```
[IC]: The intermediate conclusion.

[C]: The conclusion

<Argument with intermediate conclusions>

(1) First premise
(2) Second premise
----
(3) [IC]
(4) One more premise.
----
(5) [C]
```

Representing Dialectical Structures

- Intermediate conclusion and final conclusion defined as independent statements.
- Complex argument split into two arguments to make the intermediate conclusion visible as a node.

argdown

Map

```
[IC]: The intermediate conclusion.

[C]: The conclusion

<Argument Part 1>

(1) First premise
(2) Second premise
----
(3) [IC]

<Argument Part 2>

(1) [IC]
(2) One more premise.
----
(3) [C]
```

Dialectical Structures: Internal Structure of Individual Arguments[1]

argdown

Map

```
===
selection:
  statementSelectionMode: all # adds all statements as nodes to the map
model:
  explodeArguments: true # adds argument nodes for each inferential step of each argument
map:
  argumentLabelMode: none # no text in argument nodes
  statementLabelMode: text # no titles in statement nodes
dot:
  argument:
    shape: circle # argument nodes are in the shape of circles
    minWidth: 0.2 # let's make those circles really small
  graphVizSettings:
    rankdir: TB # arrows flow from top to bottom (to produce a tree-like structure)
===

<Argument with intermediate conclusions – one node>

(1) First premise
```

1. See https://argdown.org/guide/creating-oldschool-argument-maps-and-inference-trees.html#creating-oldschool-argument-maps-and-inference-trees

Inference Patterns & Logical Form: Simple Representation

(1) First premise
    Form: If p, then q.
(2) Second premise
    Form: p
-- from 1, 2, by Modus Ponens --
(3) *Therefore:* Conclusion

Inference Patterns & Logical Form: Detailed Representation

(1) First premise
    Form: If p, then q.
    p: _what p stands for_
    q: _what q stands for_
(2) Second premise
    Form: p
--
from 1, 2, by Modus Ponens
--
(3) *Therefore:* Conclusion

Inference Patterns & Logical Form: Machine-Readable Representation (JSON/YAML)

```
(1) First premise {formalization: "p -> q", declarations: {"p": "what p stands for", "q": "what q stands for"}}
(2) Second premise {formalization: "p"}
-- {from: ["1","2"], logic: ["modus ponens"]} --
(3) *Therefore:* Conclusion
```

👉 https://argdown.org/syntax/#statement-yaml-data



Further Links

- Argdown configuration reference: https://argdown.org/guide/configuration-cheatsheet.html
- Blog post on the transition from Argunet to Argdown: http://www.argunet.org/2018/10/26/new-beginning-introducing-argdown/